

[CS230] Sequence Models: Sequence-to-Sequence (Seq2Seq)

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1-10.7 RNN, Embedding, Debiasing - *Completed*
- **10.8 Sequence-to-Sequence (Seq2Seq)**
 - * The Problem: Different Lengths ($T_x \neq T_y$)
 - * Encoder-Decoder Architecture
 - * Context Vector (The Information Bottleneck)
 - * Teacher Forcing (Training Trick)
- 10.9 Beam Search - *Upcoming*

Chapter 11. Attention Mechanism - *Next Part*

지난 시간 복습 및 연결

지난 시간까지 우리는 단어를 벡터로 바꾸는 법 (Embedding)을 배웠습니다. 이제 단어들을 조립해 문장을 만들고, 번역하는 시스템을 만들 차례입니다. 그런데 문제가 있습니다. **입력:** "I love you" (3단어) → **출력:** "Je t'aime" (2단어). 길이가 다릅니다. 기존 RNN은 이를 처리할 수 없습니다. 이를 해결하기 위해 구글이 제안한 혁명적인 아키텍처, **Seq2Seq (Encoder-Decoder)**를 배웁니다.

1 Unit Overview

□ 핵심 목표

이 단원은 입력과 출력의 길이가 다른 시퀀스 변환 모델을 다룹니다.

- **구조:** 입력 시퀀스를 압축하는 **인코더**와, 이를 풀어내는 **디코더** 구조를 이해합니다.
- **압축:** 인코더의 마지막 은닉 상태가 문장 전체의 의미를 담은 **문맥 벡터 (Context Vector)**가 됨을 파악합니다.
- **전달:** 문맥 벡터가 디코더의 초기 상태로 전달되는 흐름을 코드로 구현합니다.
- **한계:** 문장이 길어지면 정보가 손실되는 **병목 (Bottleneck)** 현상을 인지합니다.

2 Essential Terminology

용어	역할	비유
Encoder	입력 문장을 읽고 이해함.	문장을 읽는 번역가.
Decoder	이해한 내용을 바탕으로 문장을 생성함.	번역문을 쓰는 번역가.
Context Vector	인코더가 넘겨주는 핵심 정보 (마지막 h).	머릿속에 정리된 문장의 의미.

3 Core Concepts: 다 듣고 말하기

3.1 1. The Architecture: Encoder-Decoder

Seq2Seq는 두 개의 RNN을 이어 붙인 구조입니다.

- **Encoder:** 문장 $x^{(1)} \dots x^{(T_x)}$ 를 순서대로 읽습니다. 출력을 내지 않고 은닉 상태(h)만 업데이트합니다. 마지막 상태 $h^{(T_x)}$ 에 모든 정보를 압축합니다.
- **Decoder:** 인코더가 준 문맥 벡터를 자신의 초기 상태($h_{dec}^{(0)}$)로 받습니다. 이를 바탕으로 번역문을 생성합니다.

□ 번역가의 작업 방식 (직관적 비유)

- **Read (Encoder):** 문장을 끝까지 다 읽습니다. 중간에 번역하지 않습니다. 머릿속에 핵심 의미(Context)를 정리합니다.
- **Write (Decoder):** 머릿속 의미를 바탕으로 프랑스어 문장을 처음부터 써 내려갑니다.

3.2 2. Training Trick: Teacher Forcing

학습할 때는 디코더가 실수로 엉뚱한 단어를 예측했더라도, 다음 스텝 입력으로는 **정답 단어 (Ground Truth)**를 넣어줍니다. 이를 **교사 강요 (Teacher Forcing)**라고 합니다. (초반 학습 안정화용)

4 Implementation: Keras Functional API

인코더의 상태를 디코더로 넘겨주는 핵심 코드를 작성합니다.

```
1 import tensorflow as tf
2 from tensorflow.keras.layers import Input, LSTM, Dense, Embedding
3 from tensorflow.keras.models import Model
4
5 def build_seq2seq(vocab_size, embedding_dim, latent_dim):
6
7     # --- 1. Encoder ---
8     enc_inputs = Input(shape=(None,))
9     enc_emb = Embedding(vocab_size, embedding_dim)(enc_inputs)
10
11     # return_state=True: 마지막 은닉 상태(h)와 셀 상태(c)를 반환
12     encoder_lstm = LSTM(latent_dim, return_state=True)
13     _, state_h, state_c = encoder_lstm(enc_emb)
14
15     # 문맥 벡터 (Context Vector): 문장의 모든 정보를 압축한 것
16     encoder_states = [state_h, state_c]
```

```

17
18 # --- 2. Decoder ---
19 dec_inputs = Input(shape=(None,))
20 dec_emb = Embedding(vocab_size, embedding_dim)(dec_inputs)
21
22 # return_sequences=True: 모든 타임 스텝 출력 번역문(생성)
23 decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=True)
24
25 # 핵심: 에 initial_state 인코더의 상태를 주입!
26 dec_outputs, _, _ = decoder_lstm(dec_emb, initial_state=encoder_states)
27
28 decoder_dense = Dense(vocab_size, activation='softmax')
29 dec_outputs = decoder_dense(dec_outputs)
30
31 # 모델 정의
32 model = Model([enc_inputs, dec_inputs], dec_outputs)
33 return model

```

Listing 1: Seq2Seq Model Definition

5 FAQ & Pitfalls

The Bottleneck Problem (병목 현상) (오해 방지 가이드)

Seq2Seq의 치명적 단점은 인코더가 문장이 길든 짧은 고정된 크기의 벡터 하나에 모든 정보를 우겨 넣어야 한다는 점입니다. 문장이 길어지면(50단어 이상) 정보 손실이 발생해 번역 품질이 급격히 떨어집니다. 이를 해결하기 위해 **Attention Mechanism**이 등장했습니다.

Q. 추론(Inference) 때는 어떻게 하나요?

A. 학습 때와 달리 정답을 모르므로, 디코더가 방금 예측한 단어를 다음 스텝의 입력으로 넣어주는 루프(Loop)를 직접 구현해야 합니다. (Auto-regressive)

다음 단계 (Next Step)

이제 모델을 만들었습니다. 그런데 디코더가 단어를 생성할 때, 매 순간 가장 확률 높은 단어 하나만(Greedy) 고르면 최고의 문장이 될까요? "The" 뒤에 "nice"가 올 확률이 높다고 골랐는데, 나중에 보니 "The huge building..."이 더 좋은 번역일 수도 있습니다.

다음 시간에는 번역 품질을 결정하는 결정적 탐색 알고리즘, **[Beam Search]**에 대해 알아보고, Seq2Seq의 병목 문제를 해결하는 **[Attention Mechanism]**으로 넘어가겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Structure:** Encoder가 압축하고 Decoder가 푼다.
2. **Context:** 인코더의 마지막 상태가 문장의 핵심 의미를 담은 매개체다.
3. **Transfer:** h_{enc} 를 h_{dec} 의 'initial_state' .
3. **Limit:** 긴 문장을 하나의 벡터로 압축하면 정보 손실이 발생한다.